

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет

Выпускающая кафедра: информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.01 – Информатика и вычислительная
техника

**ОТЧЕТ
ПО ТВОРЧЕСКОЙ РАБОТЕ**

На тему:

**«Автоматизация рабочего места специалиста по изучению
иностранных языков с использованием метода флеш-карточек»**

Выполнили студент группы ИВТ-25-16

Неклюдова Мария Константиновна,

студент группы ИВТ-25-16

Шулятьева Марина Евгеньевна

Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
РАЗДЕЛ 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	
1.1 Психологические основы запоминания иностранных слов методом флеш-карточек.....	5
1.2 Обзор существующих программ для изучения лексики.....	6
1.3 Сравнительный анализ аналогов(Anki,Quizlet).....	6
РАЗДЕЛ 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	
2.1 Архитектура приложения (UML-диаграммы).....	9
2.1.1 Диаграмма классов.....	9
2.1.2 Диаграмма вариантов использования.....	18
2.2 Реализация графического интерфейса.....	22
2.3 Реализация алгоритма умного повторения слов.....	26
2.4 Реализация игры "Быстрый перевод".....	29
2.5 Реализация модуля статистики и сохранения данных.....	33
ЗАКЛЮЧЕНИЕ.....	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	38

ВВЕДЕНИЕ

В современном мире изучение иностранных языков становится не просто полезным навыком, а необходимостью для профессионального роста, путешествий и общения с людьми из разных стран. Однако процесс запоминания новых слов часто вызывает трудности: человеческая память устроена так, что без регулярного повторения до 80 процентов новой информации забывается в течение первых суток. Традиционные методы заучивания — бумажные карточки, списки слов, письменные упражнения — требуют значительных временных затрат и не дают объективной обратной связи о прогрессе. Учащийся не знает, какие слова он уже выучил хорошо, а какие требуют дополнительного повторения, из-за чего эффективность обучения снижается.

Особенно остро эта проблема стоит при самостоятельном изучении языка без преподавателя. Человек может неделями учить одни и те же слова, не подозревая, что они уже давно закрепились в памяти, в то время как другие, более сложные, остаются без внимания. Бумажные карточки теряются, заметки в телефоне не систематизированы, а готовые мобильные приложения часто требуют платной подписки или содержат рекламу, отвлекающую от занятий. Кроме того, большинство существующих программ не учитывают индивидуальные особенности запоминания и не адаптируются под конкретного пользователя.

В связи с этим возникает необходимость в создании автоматизированного рабочего места специалиста по изучению иностранных слов, которое будет помогать пользователю эффективно запоминать лексику, отслеживать прогресс и автоматически корректировать частоту повторения слов в зависимости от успехов.

Целью проекта является разработка автоматизированного рабочего места специалиста по изучению иностранных языков с использованием метода флеш-карточек, которое обеспечивает создание и хранение наборов слов, интеллектуальное повторение на основе частоты ошибок, игровую механику для закрепления материала, а также ведение статистики прогресса.

Задачи проекта:

1. Провести анализ предметной области и существующих решений для изучения иностранных слов методом флеш-карточек.

2. Обосновать выбор технологий (C++, Qt, Qt Creator) для реализации автоматизированного рабочего места.
3. Спроектировать архитектуру приложения с использованием объектно-ориентированного подхода и разработать UML-диаграммы классов.
4. Реализовать графический интерфейс пользователя, включающий список наборов, карточку с вопросом, поле для ввода ответа и блок статистики.
5. Разработать алгоритм умного повторения слов, учитывающий количество ошибок и правильно данных ответов подряд.
6. Создать игровой модуль «Быстрый перевод» с ограничением по времени для закрепления материала в игровой форме.
7. Реализовать модуль статистики, отслеживающий количество дней занятий и процент выученных слов.
8. Обеспечить сохранение и загрузку пользовательских наборов слов в текстовые файлы.
9. Выполнить тестирование работоспособности программы и отладку ключевых алгоритмов.

РАЗДЕЛ 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Психологические основы запоминания иностранных слов методом флеш-карточек

Человеческая память устроена таким образом, что новая информация забывается очень быстро. Немецкий психолог Герман Эббингауз ещё в XIX веке вывел «кривую забывания», которая показывает, что без повторения человек теряет до 80 процентов новой информации в течение первых суток. Именно поэтому просто прочитать слово и его перевод один раз недостаточно для запоминания.

Метод флеш-карточек основан на принципе активного вспоминания. Вместо того чтобы пассивно смотреть на слово и перевод, пользователь сначала видит слово, пытается вспомнить перевод самостоятельно, и только потом проверяет себя. Этот процесс задействует механизмы долговременной памяти и значительно улучшает усвоение материала.

Другой важный принцип — интервальное повторение. Слова, которые забываются быстрее, нужно повторять чаще, а те, которые уже запомнились, — реже. Оптимальные интервалы повторения увеличиваются с каждым разом: через несколько минут, затем через час, через день, через неделю. В разработанной программе этот принцип реализован через алгоритм умного повторения: чем больше ошибок допускает пользователь, тем чаще слово появляется, а после нескольких правильных ответов подряд его частота снижается.

Также метод флеш-карточек использует эффект тестирования. Исследования показывают, что проверка себя (например, попытка вспомнить перевод) гораздо эффективнее для запоминания, чем простое перечитывание материала. В программе пользователь каждый раз активно вводит ответ, а не просто смотрит на правильный перевод.

Игровые элементы, добавленные в программу (ограничение времени, подсчёт очков), создают дополнительную мотивацию и снижают утомляемость при длительных занятиях. Соревновательный эффект (попытка побить свой предыдущий результат) стимулирует пользователя заниматься регулярно, что является ключевым фактором успешного запоминания.

1.2 Обзор существующих программ для изучения лексики

На сегодняшний день существует множество программ и мобильных приложений для изучения иностранных слов, каждое из которых имеет свои особенности.

Anki — одна из самых популярных программ для запоминания информации методом флеш-карточек. Её главная особенность — алгоритм интервального повторения, который автоматически рассчитывает, когда нужно показать каждую карточку. Пользователь может создавать свои наборы, добавлять изображения и аудио. Интерфейс программы сложный для новичков, но при этом она даёт много возможностей для настройки.

Quizlet — удобный сервис с красивым дизайном и несколькими режимами обучения: карточки, заучивание, письмо, подбор, тест. Есть игровые режимы. Программа популярна среди студентов, но бесплатная версия содержит рекламу, а для полноценного использования нужна платная подписка.

Memrise — программа, которая делает упор на запоминание через ассоциации и видео с носителями языка. Использует элементы геймификации: очки, уровни, турнирные таблицы. Это делает процесс обучения более увлекательным, однако большая часть контента платная.

Lingualeo — популярная в России платформа, которая предлагает не только карточки, но и тексты, аудио, видео. Есть режим «Лео-игры» для закрепления слов. Бесплатная версия сильно ограничена, некоторые функции доступны только по подписке.

Таким образом, существующие программы имеют ряд общих недостатков: сложность для начинающих пользователей, необходимость платной подписки для полноценного использования, зависимость от интернета, а также отсутствие гибкой настройки алгоритма повторения под индивидуальные особенности пользователя. Разработанная в рамках данного проекта программа призвана устранить эти недостатки: она полностью бесплатна, работает без интернета, имеет простой и понятный интерфейс и реализует собственный алгоритм умного повторения.

1.3 Сравнительный анализ аналогов(Anki,Quizlet)

Для выявления преимуществ и недостатков существующих решений был проведён сравнительный анализ двух наиболее популярных программ для изучения слов — Anki и Quizlet.

Сравнение проводилось по следующим критериям: стоимость, наличие русского интерфейса, алгоритм повторения, игровые режимы, работа без интернета, возможность создания своих наборов.

Anki является бесплатной программой с открытым исходным кодом. Она использует алгоритм интервального повторения SM-2, который считается одним из лучших для долгосрочного запоминания. Пользователь может полностью настраивать внешний вид карточек и параметры повторения. Русский интерфейс присутствует. Anki работает без интернета, так как все данные хранятся локально. Однако у программы есть существенные недостатки: сложный и непривычный интерфейс, который отпугивает новых пользователей, отсутствие игровых режимов, а мобильное приложение для iOS является платным.

Quizlet предлагает freemium-модель: базовые функции бесплатны, но за расширенные возможности (например, загрузку изображений, продвинутую статистику) нужно платить. У Quizlet есть несколько игровых режимов, которые делают обучение более увлекательным. Интерфейс простой и понятный, русский язык поддерживается. Однако алгоритм повторения в Quizlet менее точный, чем в Anki, а для полноценной работы требуется интернет, так как многие функции зависят от облачного сервера.

Критерий	Anki	Quizlet	Разработанная программа
Стоимость	Бесплатно (кроме IOS)	Freemium (есть платная подписка)	Полностью бесплатно
Русский интерфейс	Есть	Есть	Есть
Алгоритм повторения	SM-2 (сложный)	Простой	Умный (на основе ошибок)
Игровые режимы	Нет	Есть	Есть
Работа без интернета	Да	Частично	Да
Создание своих наборов	Да	Да	Да
Простота интерфейса	Низкая	Высокая	Высокая

Таблица 1- Сравнительная таблица

Вывод по сравнению. Anki имеет мощный алгоритм повторения, но сложен для новичков и не имеет игр. Quizlet прост и имеет игры, но требует интернета и платной подписки.

Разработанная программа сочетает преимущества обоих аналогов: простой интерфейс, умный алгоритм повторения (на основе количества ошибок и правильных ответов), игровой режим и полную бесплатность. При этом она работает без интернета, что является важным преимуществом для пользователей с нестабильным соединением.

Вывод по разделу 1. В теоретической части были рассмотрены психологические основы запоминания информации методом флеш-карточек, проведён обзор существующих программ для изучения лексики, а также выполнен сравнительный анализ двух наиболее популярных аналогов — Anki и Quizlet. На основе анализа были выделены ключевые требования к разрабатываемому приложению: простота интерфейса, умный алгоритм повторения с учётом ошибок, наличие игровых механик, работа без подключения к интернету и полная бесплатность. Также было обосновано использование языка C++ и фреймворка Qt как оптимальных технологий для реализации поставленных задач. Таким образом, теоретическая часть позволила сформулировать основные принципы, которые были положены в основу практической реализации автоматизированного рабочего места специалиста по изучению иностранных слов.

РАЗДЕЛ 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Архитектура приложения (UML-диаграммы)

Разработанное приложение имеет архитектуру, основанную на фреймворке Qt, с применением объектно-ориентированного программирования и механизма сигналов и слотов. Для наглядного представления структуры и работы системы созданы две UML-диаграммы: диаграмма классов, отображающая статическую организацию программы, и диаграмма последовательности, иллюстрирующая динамику одного из основных сценариев использования.

2.1.1 Диаграмма классов

Диаграмма классов отображает статическую структуру разработанного приложения: классы, их атрибуты, методы и связи между ними. Ниже на рисунке представлена полная диаграмма классов, составленная на основе реального кода проекта.



Рисунок 1-UML-диаграмма классов

Пояснение диаграммы классов:

1.Главный класс MainWindow. Класс MainWindow является главным окном приложения. Он наследуется от QMainWindow-базового класса Qt для главных окон, который предоставляет готовую структуру с центральным виджетом, панелями инструментов и областями для размещения элементов управления.

Атрибуты MainWindow:

- ✧ `currentCards:QVector<Card>`- вектор, хранящий все карточки текущего выбранного набора. Выбор вектора обусловлен эффективным доступом по индексу и быстрым добавлением новых карточек.
- ✧ `activeCards:QVector<int>`- вектор индексов ещё не выученных карточек. Позволяет быстро получать список активных слов без необходимости прохода по всем карточкам.
- ✧ `currentSetName:QString`- название текущего открытого набора. Используется для загрузки и сохранения данных, а также для отображения в интерфейсе.
- ✧ `currentCardIndex:int`- индекс текущей показываемой карточки в векторе `currentCards`.
- ✧ `lastCardIndex:int`- индекс последней показанной карточки. Необходим для работы алгоритма, исключающего повторение одного и того же слова подряд.
- ✧ `setList:QListWidget*`- указатель на виджет списка, отображающий все доступные наборы слов в левой панели.
- ✧ `questionLabel:QLabel*`- указатель на виджет текста, отображающий вопрос (слово или фразу для перевода).
- ✧ `answerInput:QLineEdit*`- указатель на поле ввода, куда пользователь вводит свой вариант перевода.
- ✧ `checkBtn:QPushButton*`- кнопка «Проверить», запускающая проверку введённого ответа.
- ✧ `dontKnowBtn:QPushButton*`- кнопка «Не знаю», показывающая правильный ответ и увеличивающая счётчик ошибок.
- ✧ `finishBtn:QPushButton*`- кнопка «Завершить», завершающая обучение по текущему набору.
- ✧ `deleteBtn:QPushButton*`- кнопка удаления пользовательского набора. Активна только для наборов, созданных самим пользователем.
- ✧ `progressBar:QProgressBar*`- полоска прогресса, отображающая процент выученных слов в текущем наборе.
- ✧ `daysLabel:QLabel*`- текстовая метка с количеством дней, когда пользователь занимался. Учитываются только уникальные дни.

- ✧ rightPanel:QWidget*- указатель на правую панель с карточкой. Используется для скрытия панели во время игры.
- ✧ gameCards:QVector<Card>- вектор карточек для игрового режима. Формируется отдельно от основного обучения.
- ✧ gameCurrentIndex : int — индекс текущей карточки в игре.
- ✧ gameScore:int- количество правильных ответов, набранных в текущей игре.
- ✧ gameTotal:int - общее количество слов в текущей игре.
- ✧ gameTimeLeft:int-оставшееся время на ответ в секундах (по умолчанию 10 секунд).
- ✧ gameTimer:QTimer*-таймер для игры «Быстрый перевод». Ограничивает время ответа.
- ✧ gameTimerLabel:QLabel*-метка, отображающая оставшееся время в игре.
- ✧ gameScoreLabel:QLabel*-метка, отображающая текущий счёт в игре.
- ✧ gameQuestionLabel:QLabel*-метка, отображающая вопрос в игре.
- ✧ gameAnswerInput : QLineEdit*-поле ввода ответа в игре.
- ✧ gameCheckBtn:QPushButton*-кнопка проверки ответа в игре.
- ✧ gameDontKnowBtn:QPushButton*-кнопка «Не знаю» в игре.
- ✧ gameWidget:QWidget*-виджет, содержащий весь интерфейс игры. Скрыт по умолчанию, появляется при запуске игры.

Методы MainWindow:

- ✧ MainWindow(parent)- конструктор класса. В нём создаются все виджеты интерфейса, загружаются стандартные наборы слов (30 английских слов), настраиваются соединения сигналов и слотов, инициализируются таймеры.
- ✧ loadEnglishToRussian()- загружает набор из 30 слов с направлением перевода «Английский → Русский».
- ✧ loadRussianToEnglish()- загружает набор из 30 слов с направлением перевода «Русский → Английский».
- ✧ loadCustomSetsFromFiles()- сканирует папку программы на наличие текстовых файлов с пользовательскими наборами и загружает их в список.

- ✧ updateActiveCards()- обновляет вектор активных карточек, исключая те, которые уже помечены как выученные.
- ✧ getNextCard()- реализует алгоритм взвешенного случайного выбора следующей карточки. Чем больше ошибок допущено при переводе слова, тем выше его вес. Последнее показанное слово получает нулевой вес для исключения повторов подряд.
- ✧ showCurrentCard()- отображает текущую карточку на экране: устанавливает текст вопроса в questionLabel, очищает поле ввода answerInput и устанавливает на него фокус.
- ✧ updateProgress()- вычисляет процент выученных слов от общего количества в наборе и обновляет значение progressBar.
- ✧ loadCustomSet(name)- загружает пользовательский набор из текстового файла. Помимо самих слов, восстанавливается накопленная статистика ошибок и правильных ответов.
- ✧ addToSetList(name)- добавляет название нового набора в список setList в левой панели.
- ✧ updateDaysCounter()- обновляет отображение количества дней занятий на основе данных из файла days_data.txt.
- ✧ saveDaysData()- сохраняет текущую дату последнего входа в файл. Если последний сохранённый день отличается от сегодняшнего, счётчик дней увеличивается на 1.
- ✧ loadDaysData()- загружает из файла сохранённое количество дней занятий и дату последнего входа.
- ✧ startGame()- запускает игру «Быстрый перевод». Пользователь выбирает количество слов для перевода, программа перемешивает выбранные карточки, запускает таймер на 10 секунд и показывает первое слово.
- ✧ onGameCheck()- проверяет введённый пользователем ответ в игре. При правильном ответе увеличивает счёт. В любом случае переходит к следующему слову.
- ✧ onGameDontKnow()- обрабатывает нажатие кнопки «Не знаю» в игре. Показывает пользователю правильный ответ и переходит к следующему слову без увеличения счёта.

- ✧ `updateGameTimer()`- обновляет отображение таймера в игре. При истечении 10 секунд показывает правильный ответ и автоматически переходит к следующему слову.
- ✧ `closeGame()`- закрывает игровой режим: скрывает `gameWidget`, останавливает таймер и показывает обратно правую панель `rightPanel` с основным обучением.

Стоты `MainWindow` (вызываются по сигналам от кнопок):

- ✧ `onSetSelected(item)`- вызывается при выборе набора из списка `setList`. Загружает выбранный набор, сбрасывает счётчики и показывает первую карточку.
- ✧ `onCheckAnswer()`- вызывается при нажатии кнопки «Проверить». Сравнивает введенный ответ с правильным без учёта регистра. При правильном ответе увеличивает `correctCount`; при достижении 7 правильных ответов подряд слово помечается как выученное. При неправильном ответе увеличивает `errors` и сбрасывает `correctCount`.
- ✧ `onDontKnow()`- вызывается при нажатии кнопки «Не знаю». Показывает правильный ответ в диалоговом окне, увеличивает счётчик ошибок для текущего слова и переходит к следующей карточке.
- ✧ `onFinish()`- вызывается при нажатии кнопки «Завершить». Завершает обучение по текущему набору, сбрасывает все счётчики и возвращает интерфейс в исходное состояние.
- ✧ `onAddOwnSet()`- вызывается при нажатии кнопки добавления набора. Открывает диалоговое окно, в котором пользователь последовательно вводит до 50 карточек (вопрос и ответ). После завершения набор сохраняется в текстовый файл и появляется в списке.
- ✧ `onDeleteSet()`- вызывается при нажатии кнопки удаления. Удаляет выбранный пользовательский набор как из списка, так и из файловой системы. Базовые наборы («Английский → Русский» и «Русский → Английский») удалить невозможно.

2.Класс Card. Класс Card является моделью данных и представляет одну флеш-карточку. Он не наследуется от Qt-классов, так как не требует сигнально-слотового взаимодействия и выполняет исключительно функцию хранения данных.

Атрибуты Card:

- ✧ question:QString- слово или фраза на изучаемом языке, которая показывается пользователю на карточке.
- ✧ answer:QString- правильный перевод или ответ, с которым программа сравнивает ввод пользователя.
- ✧ errors:int- количество ошибок, допущенных пользователем при переводе этого слова. Влияет на вес при выборе следующей карточки: чем больше ошибок, тем чаще слово появляется.
- ✧ correctCount:int - количество правильных ответов подряд. При достижении значения 7 слово помечается как выученное и удаляется из активного списка.
- ✧ learned:bool- флаг, указывающий, выучено ли слово. Выученные слова больше не показываются пользователю.

Методы Card:

- ✧ Card(q, a)- конструктор класса. Инициализирует поля question и answer переданными значениями. Поля errors и correctCount обнуляются, флаг learned устанавливается в false.

3.Стандартные классы Qt,используемые в проекте.

- ✧ QMainWindow. Класс QMainWindow является базовым классом для главных окон в Qt. Он предоставляет готовую структуру с центральным виджетом, панелями инструментов, меню, строкой состояния и док-областями. В данном проекте от него наследуется класс MainWindow.Используемые методы QMainWindow:
 - ◆ setTitle(title)- устанавливает заголовок окна.
 - ◆ resize(width, height)- задаёт размер окна.
 - ◆ setCentralWidget(widget)- устанавливает центральный виджет.
- ✧ QWidget. Класс QWidget является базовым для всех виджетов в Qt. Он предоставляет основные возможности для отображения и взаимодействия с пользователем. Где используется в проекте:
 - ◆ rightPanel- правая панель с карточкой.

- ◆ gameWidget- виджет, содержащий интерфейс игры.
- ✧ QListWidget. Класс QListWidget представляет собой виджет списка, который отображает элементы в виде списка. Используется для отображения всех доступных наборов слов в левой панели.Используемые методы:
 - ◆ addItem(item)- добавляет элемент в список.
 - ◆ clear()- очищает список.
 - ◆ count()- возвращает количество элементов.
 - ◆ item(row)- возвращает элемент по индексу.
 - ◆ takeItem(row)- удаляет элемент из списка.
- ✧ QLabel. Класс QLabel предназначен для отображения текстовой информации. Используется для вывода вопроса, статистики, дней занятий, времени и счёта в игре. Используемые методы:
 - ◆ setText(text)- устанавливает текст.
 - ◆ setAlignment(alignment)- задаёт выравнивание.
 - ◆ setWordWrap(on)- включает перенос слов.
 - ◆ setStyleSheet(style)- задаёт стиль оформления.
- ✧ QLineEdit.Класс QLineEdit представляет собой однострочное поле ввода текста. Используется для ввода ответов пользователем как в основном режиме обучения, так и в игре. Используемые методы:
 - ◆ text()- возвращает введённый текст.
 - ◆ setText(text)- устанавливает текст.
 - ◆ clear()- очищает поле.
 - ◆ setEnabled(enabled)- включает или отключает поле.
 - ◆ setPlaceholderText(text)- задаёт текст-подсказку.
 - ◆ setFocus()- устанавливает фокус ввода.
- ✧ QPushButton. Класс QPushButton представляет собой кнопку. Используется для всех кнопок интерфейса: «Проверить», «Не знаю», «Завершить», «Добавить набор», «Удалить набор», «Игра», кнопок в игре. Используемые методы:
 - ◆ setText(text)- устанавливает текст на кнопке.
 - ◆ setEnabled(enabled)- включает или отключает кнопку.
 - ◆ setStyleSheet(style)- задаёт стиль оформления.

- ✧ **QProgressBar.** Класс `QProgressBar` отображает прогресс выполнения в виде полосы. Используется для наглядного отображения процента выученных слов. Используемые методы:
 - ◆ `setValue(value)`- устанавливает текущее значение.
 - ◆ `setMinimum(value)`- устанавливает минимальное значение.
 - ◆ `setMaximum(value)`- устанавливает максимальное значение.
 - ◆ `setStyleSheet(style)`- задаёт стиль оформления.
- ✧ **QTimer.** Класс `QTimer` предоставляет таймер с поддержкой сигналов. Используется в игре «Быстрый перевод» для ограничения времени ответа (10 секунд). По истечении времени таймер генерирует сигнал `timeout()`, который вызывает слот `updateGameTimer()`. Используемые методы:
 - ◆ `start(msec)`- запускает таймер с интервалом в миллисекундах.
 - ◆ `stop()` - останавливает таймер.
- ✧ **Классы `QVBoxLayout` и `QHBoxLayout`.** Классы `QVBoxLayout` и `QHBoxLayout` являются менеджерами компоновки. `QVBoxLayout` располагает дочерние виджеты вертикально (друг под другом), а `QHBoxLayout` — горизонтально (в ряд). Используются для построения интерфейса программы. Используемые методы:
 - ◆ `addWidget(widget)`- добавляет виджет в компоновку.
 - ◆ `addLayout(layout)`- добавляет вложенную компоновку.
 - ◆ `addStretch()`- добавляет растягивающийся промежуток.
- ✧ **QFile.** Класс `QFile` предоставляет интерфейс для работы с файлами. Используется для чтения и записи пользовательских наборов слов, а также для хранения данных о днях занятий. Используемые методы:
 - ◆ `open(mode)`- открывает файл в указанном режиме.
 - ◆ `close()`- закрывает файл.
 - ◆ `remove(fileName)` — удаляет файл.
 - ◆ `exists()`- проверяет существование файла.
- ✧ **QTextStream.** Класс `QTextStream` предоставляет удобный потоковый интерфейс для чтения и записи текстовых данных в файлы. Используется для построчного чтения и записи карточек в текстовые файлы. Используемые методы:

- ◆ operator<<- запись данных в поток.
- ◆ readLine()- чтение строки из потока.
- ◆ atEnd()- проверка достижения конца файла.
- ✧ QDir (Qt). Класс QDir предоставляет доступ к файловой системе. Используется для поиска всех текстовых файлов в папке программы, чтобы загрузить пользовательские наборы. Используемые методы:
 - ◆ entryList(filters, sort)- возвращает список файлов, соответствующих фильтру.
- ✧ QMessageBox. Класс QMessageBox предназначен для отображения стандартных диалоговых окон. Используется для вывода сообщений об ошибках, уведомлений о правильных и неправильных ответах, а также для подтверждения действий (например, удаления набора). Используемые методы:
 - ◆ information(parent, title, text)- информационное окно.
 - ◆ warning(parent, title, text)- окно предупреждения.
 - ◆ question(parent, title, text, buttons)- окно с вопросом.
- ✧ QInputDialog (Qt). Класс QInputDialog предоставляет удобные диалоги для ввода данных. Используется при создании нового набора слов (ввод вопроса и ответа) и при настройке игры (выбор количества слов). Используемые методы:
 - ◆ getText(parent, title, label)- диалог для ввода текста.
 - ◆ getInt(parent, title, label, value, min, max, step)- диалог для ввода целого числа.
- ✧ QRandomGenerator. Класс QRandomGenerator используется для генерации случайных чисел. Применяется для перемешивания карточек перед началом игры и для реализации взвешенного случайного выбора следующей карточки в основном режиме обучения. Используемые методы:
 - ◆ global()- возвращает указатель на глобальный генератор.
 - ◆ bounded(max)- возвращает случайное число в диапазоне от 0 до max.
- ✧ QDate. Класс QDate предоставляет функционал для работы с календарными датами. Используется для подсчёта уникальных дней занятий: программа запоминает дату последнего входа и увеличивает

счётчик дней, если сегодняшняя дата отличается от сохранённой.

Используемые методы:

- ◆ currentDate()- возвращает текущую дату.
- ◆ toString(format)- преобразует дату в строку.
- ◆ fromString(string, format)- создаёт дату из строки.

4.Связи между классами на диаграмме.

- ✧ Наследование: MainWindow наследуется от QMainWindow. На диаграмме эта связь обозначается стрелкой, направленной от MainWindow к QMainWindow.
- ✧ Композиция: MainWindow содержит множество объектов Card в векторах currentCards и gameCards. На диаграмме эта связь обозначается стрелкой со стороны MainWindow к Card. Это означает, что при уничтожении MainWindow все содержащиеся в нём карточки также будут уничтожены.
- ✧ Ассоциация: MainWindow использует все остальные классы (QListWidget, QLabel, QLineEdit, QPushButton, QProgressBar, QTimer, QVBoxLayout, QHBoxLayout, QFile, QTextStream, QDir, QMessageBox, QInputDialog, QRandomGenerator, QDate). На диаграмме эти связи обозначаются простыми стрелками от MainWindow к соответствующим классам.

5.Соглашения об обозначениях на диаграмме.

- ✧ Знак «+» перед атрибутом или методом означает публичный доступ (public). Такие элементы доступны из любого места программы.
- ✧ Знак «-» означает приватный доступ (private). Такие элементы доступны только внутри самого класса.

2.1.2 Диаграмма последовательности

Ниже на рисунке 2 представлена диаграмма последовательности для сценария «Проверка ответа пользователя и переход к следующей карточке».

Данный сценарий является ключевым в работе приложения, так как именно в этом процессе реализуется основная логика обучения: сравнение введённого перевода с правильным ответом, обновление статистики ошибок и правильных ответов, а также автоматический переход к следующему слову.

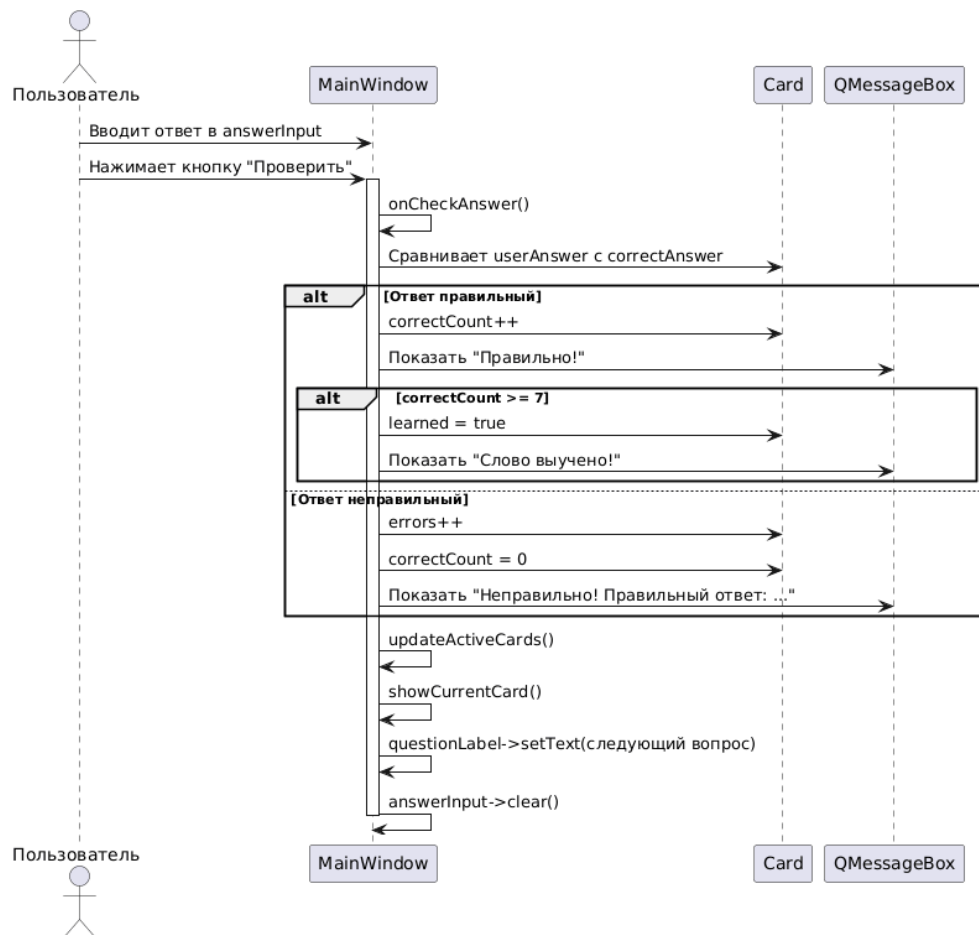


Рисунок 2- Диаграмма последовательности

Пояснение шагов диаграммы последовательности:

1. Пользователь вводит текст в поле answerInput и нажимает кнопку «Проверить» (checkBtn). Нажатие кнопки генерирует сигнал clicked(), который подключён к слоту onCheckAnswer() в классе MainWindow.
2. MainWindow вызывает метод text() у answerInput, чтобы получить введённую пользователем строку, и метод trimmed() для удаления лишних пробелов в начале и конце. Пустой ответ обрабатывается отдельно: если пользователь ничего не ввёл, QMessageBox показывает предупреждение «Введите ответ!».
3. MainWindow получает правильный ответ, обращаясь к атрибуту answer текущего объекта Card (индекс текущей карточки хранится в currentCardIndex).
4. Выполняется сравнение введённого ответа с правильным без учёта регистра с помощью метода compare(correctAnswer, Qt::CaseInsensitive).

В зависимости от результата сравнения выполняется один из двух альтернативных путей (конструкция alt на диаграмме).

5. Альтернативный путь 1 — ответ правильный:

- ✧ MainWindow вызывает `QMessageBox::information()` для отображения диалогового окна с сообщением «Правильно! Верно!».
- ✧ MainWindow вызывает метод `incrementCorrectCount()` у объекта `Card`, увеличивая счётчик правильных ответов подряд (`correctCount`).
- ✧ Проверяется условие: если `correctCount` достиг значения 7, слово считается полностью выученным. В этом случае вызывается метод `setLearned(true)` у объекта `Card`, и флаг `learned` устанавливается в `true`. Также вызывается `QMessageBox::information()` с сообщением «Отлично! Слово выучено!».

6. Альтернативный путь 2 — ответ неправильный:

- ✧ MainWindow вызывает `QMessageBox::information()` для отображения диалогового окна с сообщением «Неправильно! Правильный ответ: ...», где вместо многоточия подставляется значение `answer` из объекта `Card`.
- ✧ MainWindow вызывает метод `incrementErrors()` у объекта `Card`, увеличивая счётчик ошибок (`errors`).
- ✧ MainWindow вызывает метод `resetCorrectCount()` у объекта `Card`, обнуляя счётчик правильных ответов подряд (`correctCount = 0`).

7. После обработки ответа (независимо от того, был он правильным или неправильным) MainWindow вызывает собственный метод `updateActiveCards()`, который заново формирует вектор `activeCards`, исключая из него карточки, у которых флаг `learned` установлен в `true`.

8. MainWindow вызывает собственный метод `getNextCard()`, который реализует алгоритм взвешенного случайного выбора следующей карточки.

В этом методе:

- ✧ Для каждого индекса из вектора `activeCards` вычисляется вес: $\text{вес} = \text{errors} * 15 + 5$. Чем больше ошибок допущено при переводе слова, тем выше его вес.
- ✧ Если слово имеет 3 и более правильных ответа подряд, его вес снижается до 1.
- ✧ Последнее показанное слово (индекс хранится в `lastCardIndex`) получает вес 0, чтобы исключить повтор одного и того же слова дважды подряд.

- ✧ С помощью `QRandomGenerator::global()->bounded(totalWeight)` выбирается случайное слово с учётом весов.
 - ✧ Выбранный индекс запоминается в `lastCardIndex` и возвращается как `nextCardIndex`.
9. `MainWindow` вызывает собственный метод `showCurrentCard()`, который:
- ✧ Устанавливает текст в `questionLabel`, получая вопрос из выбранной карточки через метод `getQuestion()`.
 - ✧ Вызывает `clear()` для очистки поля `answerInput`.
 - ✧ Вызывает `setFocus()` для установки фокуса ввода в поле `answerInput`, чтобы пользователь мог сразу вводить ответ на следующее слово.
10. `MainWindow` вызывает собственный метод `updateProgress()`, который:
- ✧ Подсчитывает количество выученных слов (где `learned == true`).
 - ✧ Вычисляет процент выученных слов от общего количества в наборе.
 - ✧ Обновляет значение `progressBar` с помощью метода `setValue(percent)`.
 - ✧ Обновляет текст в `statsLabel`, отображая строку вида «Выучено: X из Y слов (Z%)».

Особенности реализации, отражённые на диаграмме:

Диаграмма последовательности показывает, как работает программа при проверке ответа пользователя. Во-первых, когда пользователь нажимает кнопку «Проверить», программа автоматически вызывает нужный метод. Это работает благодаря механизму сигналов и слотов Qt: кнопка просто сообщает, что на неё нажали, а какой именно метод вызывать — решает программа. Так кнопка не привязана жёстко к коду проверки. Во-вторых, класс `Card` только хранит данные о слове (вопрос, ответ, ошибки). Вся логика обучения собрана в классе `MainWindow`. Это упрощает код: если нужно поменять правила учёбы, меняется только один класс, а карточки остаются без изменений. В-третьих, алгоритм выбора следующего слова (`getNextCard`) написан отдельно и не зависит от интерфейса. Если потребуется изменить правила повторения слов (например, добавить учёт времени), это можно сделать в одном месте, не трогая остальной код. В-четвёртых, для всех элементов интерфейса используются стандартные классы Qt: `QLineEdit` для ввода ответа, `QPushButton` для кнопок, `QLabel` для текста, `QProgressBar` для полосы прогресса, `QMessageBox` для всплывающих окон.

2.2 Реализация графического интерфейса

Графический интерфейс приложения реализован с использованием фреймворка Qt без применения Qt Designer. Все виджеты создаются программно в конструкторе класса MainWindow, что обеспечивает полный контроль над интерфейсом и упрощает понимание кода, так как всё создание виджетов происходит в одном месте.

На рисунке 3 представлен фрагмент конструктора MainWindow, в котором создаются основные элементы интерфейса.

```
19 MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent), lastCardIndex(-1),
20   gameCurrentIndex(0), gameScore(0), gameTotal(0), gameTimeLeft(10) {
21
22   setWindowTitle("Флеш-карточки - Изучение английского");
23   resize(1000, 600);
24
25   loadDaysData();
26
27   QWidget* central = new QWidget;
28   QHBoxLayout* mainLayout = new QHBoxLayout(central);
29   mainLayout->setSpacing(20);
30
31   // Левая панель
32   QWidget* leftPanel = new QWidget;
33   QVBoxLayout* leftLayout = new QVBoxLayout(leftPanel);
34
35   QLabel* titleLabel = new QLabel("МОИ НАБОРЫ");
36   titleLabel->setStyleSheet("font-size: 16px; font-weight: bold;");
37   leftLayout->addWidget(titleLabel);
38
39   setList = new QListWidget;
40   setList->setMinimumWidth(250);
41   setList->setMaximumWidth(300);
42   leftLayout->addWidget(setList);
43
44   QPushButton* addSetBtn = new QPushButton("+ ДОБАВИТЬ СВОЙ НАБОР");
45   addSetBtn->setStyleSheet("background-color: #3498db; color: white; padding: 10px; font-weight: bold; border-radius: 5px;");
46   leftLayout->addWidget(addSetBtn);
47
48   deleteBtn = new QPushButton("- УДАЛИТЬ СВОЙ НАБОР");
49   deleteBtn->setStyleSheet("background-color: #9b59b6; color: white; padding: 10px; font-weight: bold; border-radius: 5px;");
50   deleteBtn->setEnabled(false);
51   leftLayout->addWidget(deleteBtn);
52
53   QPushButton* gameBtn = new QPushButton("ИГРА: БЫСТРЫЙ ПЕРЕВОД");
54   gameBtn->setStyleSheet("background-color: #e67e22; color: white; padding: 10px; font-weight: bold; border-radius: 5px; margin-top: 10px;");
55   leftLayout->addWidget(gameBtn);
```

Рисунок 3-Фрагмент конструктора MainWindow (создание левой панели)

В конструкторе выполняются следующие действия: установка заголовка окна и его размера 1000 на 600 пикселей, создание центрального виджета и горизонтальной компоновки с отступом 20 пикселей, создание левой панели с вертикальной компоновкой, добавление заголовка «МОИ НАБОРЫ», создание списка наборов с минимальной шириной 250 и максимальной 300 пикселей, добавление трёх кнопок с разными цветами фона и скруглёнными углами.

На рисунке 4 показан фрагмент кода, реализующий правую панель с карточкой.

```

82 // Правая панель (основная)
83 rightPanel = new QWidget;
84 QVBoxLayout rightLayout = new QVBoxLayout(rightPanel);
85
86 QGroupBox cardBox = new QGroupBox("КАРТОЧКА");
87 QVBoxLayout cardLayout = new QVBoxLayout(cardBox);
88
89 questionLabel = new QLabel("Нажмите на набор слесей");
90 questionLabel->setAlignment(Qt::AlignCenter);
91 questionLabel->setWordWrap(true);
92 questionLabel->setStyleSheet("font-size: 28px; font-weight: bold; padding: 60px; background-color: #f0f0f0; border-radius: 15px; margin: 10px;");
93 cardLayout->addWidget(questionLabel);
94
95 answerInput = new QLineEdit;
96 answerInput->setPlaceholderText("Введите ответ здесь...");
97 answerInput->setEnabled(false);
98 answerInput->setStyleSheet("font-size: 18px; padding: 15px; border: 2px solid #ccc; border-radius: 10px; margin: 10px;");
99 cardLayout->addWidget(answerInput);
100
101 QHBoxLayout btnLayout = new QHBoxLayout;
102 btnLayout->setSpacing(15);
103
104 checkBtn = new QPushButton("Проверить");
105 dontKnowBtn = new QPushButton("Не знаю");
106 finishBtn = new QPushButton("Завершить");
107
108 checkBtn->setStyleSheet("background-color: #27ae60; color: white; padding: 12px; font-weight: bold; border-radius: 8px;");
109 dontKnowBtn->setStyleSheet("background-color: #e74c3c; color: white; padding: 12px; font-weight: bold; border-radius: 8px;");
110 finishBtn->setStyleSheet("background-color: #95a5a6; color: white; padding: 12px; font-weight: bold; border-radius: 8px;");
111
112 checkBtn->setEnabled(false);
113 dontKnowBtn->setEnabled(false);
114 finishBtn->setEnabled(false);
115
116 btnLayout->addWidget(checkBtn);
117 btnLayout->addWidget(dontKnowBtn);
118 btnLayout->addWidget(finishBtn);
119 cardLayout->addLayout(btnLayout);
120
121 rightLayout->addWidget(cardBox);
122
123 mainLayout->addWidget(leftPanel, 1);
124 mainLayout->addWidget(rightPanel, 3);

```

Рисунок 4-Фрагмент конструктора MainWindow (создание правой панели)

Здесь создаётся правая панель с группой «КАРТОЧКА», внутри которой расположены: метка для вопроса с крупным шрифтом 28 пикселей, жирным начертанием, отступами 60 пикселей и серым фоном; поле для ввода ответа с шрифтом 18 пикселей и рамкой; три кнопки «Проверить», «Не знаю», «Завершить» с цветами зелёный, красный и серый соответственно. Изначально поле ввода и кнопки отключены (setEnabled(false)), они активируются только после выбора набора. Левая и правая панели добавляются в главную компоновку с соотношением размеров 1 к 3.

На рисунке 5 показана настройка внешнего вида карточки с вопросом.

```

89 questionLabel = new QLabel("Нажмите на набор слесей");
90 questionLabel->setAlignment(Qt::AlignCenter);
91 questionLabel->setWordWrap(true);
92 questionLabel->setStyleSheet("font-size: 28px; font-weight: bold; padding: 60px; background-color: #f0f0f0; border-radius: 15px; margin: 10px;");
93 cardLayout->addWidget(questionLabel);

```

Рисунок 5- Настройка внешнего вида метки вопроса

Метод setAlignment(Qt::AlignCenter) центрирует текст по горизонтали и вертикали. Метод setWordWrap(true) включает перенос слов, если текст вопроса не помещается в одну строку. Метод setStyleSheet() задаёт CSS-стили: размер шрифта 28 пикселей, жирное начертание, внутренние отступы 60 пикселей, серый фон, скругление углов на 15 пикселей и внешние отступы 10 пикселей.

На рисунке 6 показана настройка поля для ввода ответа.

```

95 answerInput = new QLineEdit;
96 answerInput->setPlaceholderText("Введите ответ здесь...");
97 answerInput->setEnabled(false);
98 answerInput->setStyleSheet("font-size: 18px; padding: 15px; border: 2px solid #ccc; border-radius: 10px; margin: 10px;");
99 cardLayout->addWidget(answerInput);

```

Рисунок 6- Настройка поля для ввода ответа

Метод setPlaceholderText() задаёт серый текст-подсказку, который отображается внутри пустого поля. Метод setEnabled(false) делает поле недоступным для ввода до выбора набора.

Стилизация включает шрифт 18 пикселей, внутренние отступы 15 пикселей, серую рамку толщиной 2 пикселя и скругление углов на 10 пикселей.

На рисунке 7 показана реализация прогресс-бара и блока статистики.

```

57 // Статистика
58 QGroupBox* statsGroup = new QGroupBox("");
59 QVBoxLayout* statsLayout = new QVBoxLayout(statsGroup);
60
61 daysLabel = new QLabel();
62 daysLabel->setStyleSheet("font-size: 14px; font-weight: bold; color: #2c3e50; padding: 5px;");
63 statsLayout->addWidget(daysLabel);
64
65 progressLabel = new QLabel("Прогресс:");
66 progressLabel->setStyleSheet("font-size: 12px; margin-top: 10px;");
67 statsLayout->addWidget(progressLabel);
68
69 progressBar = new QProgressBar;
70 progressBar->setMinimum(0);
71 progressBar->setMaximum(100);
72 progressBar->setValue(0);
73 statsLayout->addWidget(progressBar);
74
75 statsLabel = new QLabel("Выберите набор");
76 statsLabel->setStyleSheet("font-size: 12px; color: #7f8c8d; margin-top: 5px;");
77 statsLayout->addWidget(statsLabel);
78
79 leftLayout->addWidget(statsGroup);
80 leftLayout->addStretch();

```

Рисунок 7-Реализация прогресс-бара и блока статистики

Блок статистики включает метку с количеством дней занятий, полосу прогресса, отображающую процент выученных слов, и текстовую метку с информацией о количестве выученных слов. Прогресс-бар имеет диапазон от 0 до 100 процентов, серую рамку и зелёную заливку.

На рисунке 8 показан фрагмент кода, создающий игровой интерфейс.

```

126 // Создание игров
127 gameWidget = new QWidget(this);
128 QVBoxLayout* gameLayout = new QVBoxLayout(gameWidget);
129
130 QLabel* gameTitle = new QLabel("WPA: БЫСТРЫЙ ПЕРЕВОД");
131 gameTitle->setAlignment(Qt::AlignCenter);
132 gameTitle->setStyleSheet("font-size: 24px; font-weight: bold; color: #e67e22; margin: 20px;");
133 gameLayout->addWidget(gameTitle);
134
135 gameTimerLabel = new QLabel("Время: 10 сек");
136 gameTimerLabel->setAlignment(Qt::AlignCenter);
137 gameTimerLabel->setStyleSheet("font-size: 18px; font-weight: bold; color: #e74c3c;");
138 gameLayout->addWidget(gameTimerLabel);
139
140 gameScoreLabel = new QLabel("Счёт: 0");
141 gameScoreLabel->setAlignment(Qt::AlignCenter);
142 gameScoreLabel->setStyleSheet("font-size: 16px;");
143 gameLayout->addWidget(gameScoreLabel);
144
145 gameQuestionLabel = new QLabel();
146 gameQuestionLabel->setAlignment(Qt::AlignCenter);
147 gameQuestionLabel->setStyleSheet("font-size: 28px; font-weight: bold; padding: 40px; background-color: #f0f0f0; border-radius: 15px; margin: 20px;");
148 gameQuestionLabel->setWordWrap(true);
149 gameLayout->addWidget(gameQuestionLabel);
150
151 gameAnswerInput = new QLineEdit;
152 gameAnswerInput->setPlaceholderText("Введите ответ...");
153 gameAnswerInput->setStyleSheet("font-size: 18px; padding: 15px; border: 2px solid #ccc; border-radius: 10px; margin: 10px;");
154 gameLayout->addWidget(gameAnswerInput);
155
156 QHBoxLayout* gameBtnLayout = new QHBoxLayout;
157 gameCheckBtn = new QPushButton("ПОПРАВИТЬ");
158 gameDontKnowBtn = new QPushButton("НЕ ЗНАЮ");
159 gameCheckBtn->setStyleSheet("background-color: #27ae60; color: white; padding: 12px; font-weight: bold; font-size: 14px; border-radius: 8px;");
160 gameDontKnowBtn->setStyleSheet("background-color: #e74c3c; color: white; padding: 12px; font-weight: bold; font-size: 14px; border-radius: 8px;");
161 gameBtnLayout->addWidget(gameCheckBtn);
162 gameBtnLayout->addWidget(gameDontKnowBtn);
163 gameLayout->addLayout(gameBtnLayout);
164
165 QPushButton* gameCloseBtn = new QPushButton("ЗАКРЫТЬ ИГРУ");
166 gameCloseBtn->setStyleSheet("background-color: #95a5a6; color: white; padding: 10px; margin-top: 20px; font-weight: bold; border-radius: 8px;");
167 gameLayout->addWidget(gameCloseBtn);
168
169 mainLayout->addWidget(gameWidget);
170 gameWidget->hide();
171
172 gameTimer = new QTimer(this);
173
174 setCentralWidget(central);

```

Рисунок 8-Фрагмент конструктора MainWindow (создание игрового интерфейса)

Игровой интерфейс включает: заголовок оранжевого цвета, метку таймера красного цвета, метку счёта, метку для вопроса с серым фоном и

крупным шрифтом, поле для ввода ответа, две кнопки «ПРОВЕРИТЬ» (зелёная) и «НЕ ЗНАЮ» (красная), а также кнопку «ЗАКРЫТЬ ИГРУ» серого цвета. Весь игровой виджет по умолчанию скрыт (`gameWidget->hide()`) и появляется только после нажатия кнопки «ИГРА: БЫСТРЫЙ ПЕРЕВОД».

На рисунке 9 показан метод обновления прогресса.

```
529 void MainWindow::updateProgress() {
530     if (currentCards.isEmpty()) {
531         progressBar->setValue(0);
532         statsLabel->setText("Выберите набор");
533         return;
534     }
535
536     int total = currentCards.size();
537     int learned = 0;
538     for (int i = 0; i < total; i++) {
539         if (currentCards[i].learned) learned++;
540     }
541
542     int percent = (total > 0) ? (learned * 100 / total) : 0;
543     progressBar->setValue(percent);
544     statsLabel->setText(QString("Выучено: %1 из %2 слов (%3%)").arg(learned).arg(total).arg(percent));
545 }
```

Рисунок 9- Метод обновления прогресса

Метод проверяет, пуст ли текущий набор. Если набор пуст, прогресс-бар сбрасывается в 0, а текстовая метка устанавливается в «Выберите набор». Если набор содержит карточки, метод подсчитывает количество выученных слов (где флаг `learned` равен `true`), вычисляет процент выученных слов от общего количества, обновляет значение прогресс-бара с помощью `setValue(percent)` и выводит в текстовую метку строку вида «Выучено: X из Y слов (Z%)». Цвет прогресс-бара остаётся зелёным, как было задано в конструкторе при создании виджета.

На рисунке 10 показан метод обновления счётчика дней занятий.

```
547 void MainWindow::updateDaysCounter() {
548     loadDaysData();
549
550     int totalDays = 0;
551     QFile file("days_data.txt");
552     if (file.open(QIODevice::ReadOnly)) {
553         QTextStream in(&file);
554         if (!in.atEnd()) {
555             totalDays = in.readLine().toInt();
556         }
557         file.close();
558     }
559
560     daysLabel->setText(QString("Дней занятий: %1").arg(totalDays));
561 }
```

Рисунок 10- Метод обновления счётчика дней занятий

Метод загружает из файла days_data.txt сохранённое количество дней, когда пользователь запускал программу, и отображает это значение в левой панели под блоком статистики.

2.3 Реализация алгоритма умного повторения слов

Алгоритм умного повторения слов является ключевой частью приложения. Он определяет, какое слово показывать пользователю следующим, основываясь на статистике ошибок и правильных ответов. Чем больше ошибок допущено при переводе слова, тем чаще оно появляется. Если пользователь отвечает правильно несколько раз подряд, частота появления слова снижается, а после 7 правильных ответов слово считается полностью выученным и удаляется из активного списка.

На рисунке 11 показан метод getNextCard(), реализующий взвешенный случайный выбор следующей карточки.

```
285 int MainWindow::getNextCard() {
286     if (activeCards.isEmpty()) return -1;
287
288     if (activeCards.size() == 1) {
289         return activeCards[0];
290     }
291
292     QVector<int> weights;
293     int totalWeight = 0;
294
295     for (int i = 0; i < activeCards.size(); i++) {
296         int idx = activeCards[i];
297         int weight = currentCards[idx].errors * 15 + 5;
298         if (currentCards[idx].correctCount >= 3) {
299             weight = 1;
300         }
301         if (idx == lastCardIndex) {
302             weight = 0;
303         }
304         weights.append(weight);
305         totalWeight += weight;
306     }
307
308     if (totalWeight == 0) {
309         for (int i = 0; i < activeCards.size(); i++) {
310             int idx = activeCards[i];
311             if (idx != lastCardIndex) {
312                 return idx;
313             }
314         }
315         return activeCards[0];
316     }
317
318     int random = QRandomGenerator::global()->bounded(totalWeight);
319     int sum = 0;
320     for (int i = 0; i < weights.size(); i++) {
321         sum += weights[i];
322         if (random < sum) {
323             return activeCards[i];
324         }
325     }
326     return activeCards[0];
327 }
328
```

Рисунок 11- Метод getNextCard()- взвешенный случайный выбор следующей карточки

Метод работает следующим образом. Если нет активных карточек, возвращается -1. Если активна только одна карточка, она и возвращается. Далее для каждой активной карточки вычисляется вес. Базовый вес рассчитывается по формуле: вес = количество ошибок × 15 + 5. Чем больше ошибок, тем выше вес,

а значит, тем выше вероятность выбора этого слова. Если слово имеет 3 и более правильных ответа подряд, его вес снижается до 1, так как слово уже начинает запоминаться. Если слово было показано последним (`idx == lastCardIndex`), его вес обнуляется, чтобы одно и то же слово не повторялось дважды подряд. Затем вычисляется общая сумма весов, и генерируется случайное число от 0 до `totalWeight`. После этого выбирается карточка, на которую выпало случайное число. Если все веса оказались равны нулю (например, осталось только два слова, которые оба были показаны), метод возвращает любое слово, кроме последнего показанного.

На рисунке 12 показан метод `onCheckAnswer()`, который обрабатывает ответ пользователя и обновляет статистику карточки.

```

347 void MainWindow::onCheckAnswer() {
348     QString userAnswer = answerInput->text().trimmed();
349     if (userAnswer.isEmpty()) {
350         QMessageBox::warning(this, "Внимание", "Введите ответ!");
351         return;
352     }
353
354     QString correctAnswer = currentCards[currentCardIndex].answer;
355     int
356     if (userAnswer.compare(correctAnswer, Qt::CaseInsensitive) == 0) {
357         QMessageBox::information(this, "Правильно!", "Верно!");
358         currentCards[currentCardIndex].correctCount++;
359
360         if (currentCards[currentCardIndex].correctCount >= 7) {
361             currentCards[currentCardIndex].learned = true;
362             QMessageBox::information(this, "Отлично!", "Слово выучено!");
363         }
364         updateActiveCards();
365         showCurrentCard();
366     } else {
367         QMessageBox::information(this, "Ошибка!", "Неправильно! Правильный ответ: " + correctAnswer);
368         currentCards[currentCardIndex].errors++;
369         currentCards[currentCardIndex].correctCount = 0;
370         updateActiveCards();
371         showCurrentCard();
372     }
373 }

```

Рисунок 12- Метод `onCheckAnswer()`- обработка ответа пользователя

Метод сначала получает введенный пользователем ответ и удаляет лишние пробелы с помощью `trimmed()`. Если поле ввода пустое, выводится предупреждение. Затем метод сравнивает ответ пользователя с правильным ответом без учёта регистра с помощью `compare(correctAnswer, Qt::CaseInsensitive)`.

Если ответ правильный, показывается диалоговое окно «Правильно! Верно!», увеличивается счётчик `correctCount` у текущей карточки. Если `correctCount` достигает 7, слово помечается как выученное (`learned = true`), и показывается окно «Отлично! Слово выучено!».

Если ответ неправильный, показывается диалоговое окно с сообщением «Неправильно! Правильный ответ: ...», увеличивается счётчик ошибок errors, а счётчик правильных ответов подряд correctCount сбрасывается в 0.

После обработки ответа (в любом случае) вызывается метод updateActiveCards(), который обновляет список активных карточек, исключая выученные, и метод showCurrentCard(), который показывает следующую карточку, выбранную алгоритмом getNextCard().

На рисунке 13 показан метод updateActiveCards(), который формирует список активных (невыученных) карточек.

```
276 void MainWindow::updateActiveCards() {
277     activeCards.clear();
278     for (int i = 0; i < currentCards.size(); i++) {
279         if (!currentCards[i].learned) {
280             activeCards.append(i);
281         }
282     }
283 }
```

Рисунок 13- Метод updateActiveCards()- обновление списка активных карточек

Метод очищает вектор activeCards и проходит по всем карточкам в векторе currentCards.

Если карточка не помечена как выученная (learned == false), её индекс добавляется в activeCards. Таким образом, выученные слова больше не участвуют в выборе и не показываются пользователю.

На рисунке 14 показан метод onDontKnow(), который обрабатывает нажатие кнопки «Не знаю».

```
375 void MainWindow::onDontKnow() {
376     QString correctAnswer = currentCards[currentCardIndex].answer;
377     QMessageBox::information(this, "Правильный ответ", "Правильный ответ: " + correctAnswer + " Слово будет повторяться чаще!");
378     currentCards[currentCardIndex].errors++;
379     currentCards[currentCardIndex].correctCount = 0;
380     updateActiveCards();
381     showCurrentCard();
382 }
```

Рисунок 14- Метод onDontKnow()- обработка кнопки «Не знаю»

При нажатии кнопки «Не знаю» метод показывает правильный ответ в диалоговом окне, увеличивает счётчик ошибок errors на 1, обнуляет счётчик правильных ответов подряд correctCount, обновляет список активных карточек и показывает следующее слово. Это гарантирует, что слова, которые пользователь не знает, будут появляться чаще благодаря увеличенному весу.

2.4 Реализация игры "Быстрый перевод"

Игра «Быстрый перевод» является дополнительной механикой, предназначенной для закрепления изученных слов в игровой форме с ограничением по времени. Пользователь выбирает количество слов для перевода, после чего программа перемешивает выбранные карточки и показывает их по одной. На каждое слово даётся 10 секунд. Если пользователь успевает ответить правильно, начисляется очко. Если время истекло или пользователь нажимает кнопку «Не знаю», игра автоматически переходит к следующему слову, не начисляя очко. В конце игры показывается результат: количество правильных ответов и процент.

На рисунке 15 показан метод `startGame()`, который запускает игру.

```

608 void MainWindow::startGame() {
609     if (currentCards.isEmpty()) {
610         QMessageBox::warning(this, "Ошибка", "Сначала выберите набор для изучения!");
611         return;
612     }
613
614     bool ok;
615     int wordCount = QInputDialog::getInt(this, "Настройки игры",
616         "Сколько слов вы хотите перевести? (Максимум: " + QString::number(currentCards.size()) + ")",
617         5, 1, currentCards.size(), 1, &ok);
618
619     if (!ok) return;
620
621     gameCards.clear();
622     QVector<int> indices;
623     for (int i = 0; i < currentCards.size(); i++) {
624         indices.append(i);
625     }
626     std::shuffle(indices.begin(), indices.end(), std::mt19937(std::random_device()));
627
628     for (int i = 0; i < wordCount; i++) {
629         gameCards.append(currentCards[indices[i]]);
630     }
631
632     gameCurrentIndex = 0;
633     gameScore = 0;
634     gameTotal = gameCards.size();
635     gameTimeLeft = 10;
636
637     gameScoreLabel->setText(QString("Счет: 0 / %1").arg(gameTotal));
638     gameTimerLabel->setText("Время: 10 сек");
639     gameQuestionLabel->setText(gameCards[0].question);
640     gameAnswerInput->clear();
641     gameAnswerInput->setEnabled(true);
642     gameCheckBtn->setEnabled(true);
643     gameDontKnowBtn->setEnabled(true);
644
645     rightPanel->hide();
646     gameWidget->show();
647     gameTimer->start(1000);
648 }

```

Рисунок 15- Метод `startGame()`- запуск игры

Метод сначала проверяет, выбран ли какой-либо набор. Если набор не выбран, выводится предупреждение. Затем с помощью `QInputDialog::getInt()` пользователь выбирает количество слов для перевода от 1 до общего количества слов в наборе. Для перемешивания карточек создаётся вектор индексов `indices`, который заполняется числами от 0 до размера набора. Функция `std::shuffle` перемешивает этот вектор, после чего

из currentCards в gameCards добавляются первые wordCount карточек в случайном порядке.

Далее инициализируются переменные игры: gameCurrentIndex устанавливается в 0, gameScore в 0, gameTotal- количество выбранных слов, gameTimeLeft- 10 секунд. Обновляются элементы интерфейса: метка счёта, метка времени, метка с вопросом.

Поле ввода и кнопки становятся активными. Правая панель с основным обучением скрывается (rightPanel->hide()), показывается виджет игры (gameWidget->show()), и запускается таймер с интервалом 1000 миллисекунд (1 секунда).

На рисунке 16 показан метод onGameCheck(), который проверяет ответ в игре.

```
650 void MainWindow::onGameCheck() {
651     QString userAnswer = gameAnswerInput->text().trimmed();
652     if (userAnswer.isEmpty()) {
653         QMessageBox::warning(this, "Внимание", "Введите ответ!");
654         return;
655     }
656
657     QString correctAnswer = gameCards[gameCurrentIndex].answer;
658
659     if (userAnswer.compare(correctAnswer, Qt::CaseInsensitive) == 0) {
660         gameScore++;
661         gameScoreLabel->setText(QString("Счет: %1 / %2").arg(gameScore).arg(gameTotal));
662         QMessageBox::information(this, "Правильно!", "Верно!");
663     } else {
664         QMessageBox::information(this, "Ошибка!", "Неправильно! Правильный ответ: " + correctAnswer);
665     }
666
667     gameCurrentIndex++;
668
669     if (gameCurrentIndex >= gameTotal) {
670         gameTimer->stop();
671         QMessageBox::information(this, "Игра окончена",
672             QString("Игра завершена! Правильных ответов: %1 из %2 Процент: %3%")
673                 .arg(gameScore).arg(gameTotal).arg(gameScore * 100 / gameTotal));
674         closeGame();
675     } else {
676         gameTimeLeft = 10;
677         gameTimerLabel->setText("Время: 10 сек");
678         gameQuestionLabel->setText(gameCards[gameCurrentIndex].question);
679         gameAnswerInput->clear();
680         gameAnswerInput->setEnabled(true);
681         gameCheckBtn->setEnabled(true);
682         gameDontKnowBtn->setEnabled(true);
683         gameTimer->start(1000);
684     }
685 }
```

Рисунок 16- Метод onGameCheck()- проверка ответа в игре

Метод получает ответ пользователя и удаляет лишние пробелы. Если поле ввода пустое, выводится предупреждение. Ответ сравнивается с правильным без учёта регистра. При правильном ответе увеличивается счётчик gameScore, обновляется метка счёта и показывается окно «Правильно». При неправильном ответе показывается окно «Ошибка» с правильным ответом.

После проверки индекс текущей карточки увеличивается. Если все слова пройдены, таймер останавливается, показывается окно с результатами игры (количество правильных ответов и процент), и игра закрывается вызовом `closeGame()`.

Если остались ещё слова, таймер сбрасывается до 10 секунд, метка времени обновляется, показывается следующий вопрос, поле ввода очищается и снова становится активным, и таймер запускается заново.

На рисунке 17 показан метод `onGameDontKnow()`, который обрабатывает нажатие кнопки «Не знаю» в игре.

```

687 void MainWindow::onGameDontKnow() {
688     QString correctAnswer = gameCards[gameCurrentIndex].answer;
689     QMessageBox::information(this, "Правильный ответ", "Правильный ответ: " + correctAnswer);
690
691     gameCurrentIndex++;
692
693     if (gameCurrentIndex >= gameTotal) {
694         gameTimer->stop();
695         QMessageBox::information(this, "Игра окончена",
696             QString("Игра завершена! Правильных ответов: %1 из %2 Процент: %3%")
697                 .arg(gameScore).arg(gameTotal).arg(gameScore * 100 / gameTotal));
698         closeGame();
699     } else {
700         gameTimeLeft = 10;
701         gameTimerLabel->setText("Время: 10 сек");
702         gameQuestionLabel->setText(gameCards[gameCurrentIndex].question);
703         gameAnswerInput->clear();
704         gameAnswerInput->setEnabled(true);
705         gameCheckBtn->setEnabled(true);
706         gameDontKnowBtn->setEnabled(true);
707         gameTimer->start(1000);
708     }
709 }

```

Рисунок 17-Метод `onGameDontKnow()`- обработка кнопки «Не знаю» в игре

При нажатии кнопки «Не знаю» метод показывает правильный ответ в диалоговом окне, после чего индекс текущей карточки увеличивается. Далее логика аналогична методу `onGameCheck()`: если слова закончились — игра завершается, если нет- показывается следующее слово, таймер сбрасывается.

На рисунке 18 показан метод `updateGameTimer()`, который обновляет таймер игры.

```

711 void MainWindow::updateGameTimer() {
712     gameTimeLeft--;
713     gameTimerLabel->setText(QString("Время: %1 сек").arg(gameTimeLeft));
714
715     if (gameTimeLeft <= 0) {
716         gameTimer->stop();
717
718         QMessageBox::information(this, "Время вышло!",
719             "Время на ответ истекло! Правильный ответ: " + gameCards[gameCurrentIndex].answer);
720
721         gameCurrentIndex++;
722
723         if (gameCurrentIndex >= gameTotal) {
724             QMessageBox::information(this, "Игра окончена",
725                 QString("Игра завершена! Правильных ответов: %1 из %2 Процент: %3%")
726                     .arg(gameScore).arg(gameTotal).arg(gameScore * 100 / gameTotal));
727             closeGame();
728         } else {
729             gameTimeLeft = 10;
730             gameTimerLabel->setText("Время: 10 сек");
731             gameQuestionLabel->setText(gameCards[gameCurrentIndex].question);
732             gameAnswerInput->clear();
733             gameAnswerInput->setEnabled(true);
734             gameCheckBtn->setEnabled(true);
735             gameDontKnowBtn->setEnabled(true);
736             gameTimer->start(1000);
737         }
738     }
739 }

```

Рисунок 18- Метод `updateGameTimer()`- обновление таймера игры

Метод вызывается каждую секунду по сигналу таймера. Он уменьшает gameTimeLeft на 1 и обновляет метку времени. Если время истекло (gameTimeLeft <= 0), таймер останавливается, показывается окно «Время вышло!» с правильным ответом. Далее индекс текущей карточки увеличивается, и игра либо завершается, либо переходит к следующему слову сбросом таймера до 10 секунд.

На рисунке 19 показан метод closeGame(), который закрывает игру и возвращает пользователя к основному обучению.

```
741 void MainWindow::closeGame() {  
742     gameWidget->hide();  
743     gameTimer->stop();  
744     rightPanel->show();  
745 }
```

Рисунок 19- Метод closeGame()- закрытие игры

Метод скрывает виджет игры (gameWidget->hide()), останавливает таймер и показывает обратно правую панель с основным обучением (rightPanel->show()). Это позволяет пользователю вернуться к обычному режиму изучения карточек после завершения игры.

2.5 Реализация модуля статистики и сохранения данных

Для обеспечения сохранности пользовательских данных между сеансами работы программы реализован механизм сохранения наборов слов в текстовые файлы. Все созданные пользователем наборы сохраняются в папке программы в виде отдельных .txt файлов, названных по имени набора.

На рисунке 20 показан метод onAddOwnSet(), который создаёт и сохраняет новый набор в файл.


```

409 void MainWindow::onAddOwnSet() {
410     bool ok;
411     QString setName = QInputDialog::getText(this, "Новый набор", "Введите название набора (до 50 карточек):", QLineEdit::Normal, "", &ok);
412     if (!ok || setName.isEmpty()) return;
413     QVector<Card> newCards;
414     int cardCount = 0;
415     while (cardCount < 50) {
416         QString question = QInputDialog::getText(this, QString("Карточка %1 из 50").arg(cardCount + 1), "Введите слово или вопрос (Cancel - закончить):", QLineEdit::Normal, "", &ok);
417         if (!ok || question.isEmpty()) break;
418         QString answer = QInputDialog::getText(this, "Ответ", "Введите перевод или ответ:", QLineEdit::Normal, "", &ok);
419         if (!ok || answer.isEmpty()) {
420             QMessageBox::warning(this, "Ошибка", "Ответ не может быть пустым!");
421             continue;
422         }
423         newCards.append(Card(question, answer));
424         cardCount++;
425     }
426     if (newCards.isEmpty()) {
427         QMessageBox::warning(this, "Ошибка", "Набор не может быть пустым!");
428         return;
429     }
430     QFile file(setName + ".txt");
431     if (file.open(QIODevice::WriteOnly)) {
432         QTextStream out(&file);
433         out << newCards.size() << "\n";
434         for (int i = 0; i < newCards.size(); i++) {
435             out << newCards[i].question << "\n";
436             out << newCards[i].answer << "\n";
437             out << "0\n";
438             out << "0\n";
439             out << "0\n";
440         }
441         file.close();
442     }
443     QListWidgetItem item = new QListWidgetItem(setName);
444     item->setData(Qt::UserRole, "custom");
445     setList->addItem(item);
446     QMessageBox::information(this, "Успех", QString("Набор '%1' создан! Добавлено карточек: %2").arg(setName).arg(cardCount));
447 }

```

Рисунок 20- Метод onAddOwnSet()-создание и сохранение пользовательского набора

После того как пользователь ввёл название набора и все карточки, создаётся файл с именем setName + ".txt". В первой строке файла записывается количество карточек в наборе. Затем для каждой карточки последовательно записываются: вопрос, ответ, количество ошибок (0), количество правильных ответов подряд (0), флаг выученности (0). Все пользовательские наборы сохраняются в ту же папку, где находится исполняемый файл программы.

На рисунке 21 показан метод loadCustomSet(), который загружает выбранный пользователем набор из файла.

```

504 void MainWindow::loadCustomSet(QString name) {
505     currentCards.clear();
506     QFile file(name + ".txt");
507     if (file.open(QIODevice::ReadOnly)) {
508         QTextStream in(&file);
509         int size = in.readLine().toInt();
510         for (int i = 0; i < size; i++) {
511             QString q = in.readLine();
512             if (q.isEmpty()) break;
513             QString a = in.readLine();
514             int err = in.readLine().toInt();
515             int corr = in.readLine().toInt();
516             bool learned = (in.readLine() == "1");
517             Card c(q, a);
518             c.errors = err;
519             c.correctCount = corr;
520             c.learned = learned;
521             currentCards.append(c);
522         }
523         file.close();
524     }
525 }

```

Рисунок 21- Метод loadCustomSet()- загрузка набора из файла

Метод вызывается при выборе пользователем набора из списка. Он открывает файл с именем name + ".txt", читает количество карточек, а затем

последовательно для каждой карточки считывает вопрос, ответ, количество ошибок, количество правильных ответов подряд и флаг выученности. Эти данные восстанавливаются в объекте Card, который добавляется в вектор currentCards. Таким образом, при повторном открытии программы пользователь может выбрать ранее созданный набор из списка (названия наборов отображаются в левой панели) и продолжить обучение с того же места, где остановился, с сохранённой статистикой ошибок.

На рисунке 22 показан метод onDeleteSet(), который удаляет пользовательский набор.

```

458 void MainWindow::onDeleteSet() {
459     if (currentSetName.isEmpty()) {
460         QMessageBox::warning(this, "Ошибка", "Выберите набор для удаления!");
461         return;
462     }
463
464     if (currentSetName == "АНГЛИЙСКИЙ -> РУССКИЙ (30 слов)" || currentSetName == "РУССКИЙ -> АНГЛИЙСКИЙ (30 слов)") {
465         QMessageBox::warning(this, "Ошибка", "Нельзя удалить базовый набор!");
466         return;
467     }
468
469     QMessageBox msgBox;
470     msgBox.setWindowTitle("Удалить набор");
471     msgBox.setText(QString("Вы уверены, что хотите удалить набор '%1'?").arg(currentSetName));
472     msgBox.setStandardButtons(QMessageBox::Yes | QMessageBox::No);
473     msgBox.setButtonText(QMessageBox::Yes, "Да");
474     msgBox.setButtonText(QMessageBox::No, "Нет");
475
476     int reply = msgBox.exec();
477
478     if (reply == QMessageBox::Yes) {
479         QFile::remove(currentSetName + ".txt");
480
481         for (int i = 0; i < setList->count(); i++) {
482             QListWidgetItem* item = setList->item(i);
483             if (item->text() == currentSetName) {
484                 delete setList->takeItem(i);
485                 break;
486             }
487         }
488
489         currentCards.clear();
490         activeCards.clear();
491         currentSetName = "";
492         lastCardIndex = -1;
493         questionLabel->setText("Выберите набор для изучения");
494         answerInput->setEnabled(false);
495         checkBtn->setEnabled(false);
496         dontKnowBtn->setEnabled(false);
497         finishBtn->setEnabled(false);
498         deleteBtn->setEnabled(false);
499         updateProgress();
500         QMessageBox::information(this, "Успех", "Набор удален!");
501     }
502 }

```

Рисунок 22- Метод onDeleteSet()- удаление пользовательского набора

Перед удалением метод проверяет, выбран ли набор, и запрещает удаление базовых наборов (английского и русского). Затем выводится диалог подтверждения с кнопками «Да» и «Нет». При подтверждении файл набора удаляется с помощью QFile::remove(), набор удаляется из списка в интерфейсе, а все переменные текущего набора сбрасываются. Базовые наборы защищены от случайного удаления.

На рисунке 23 показаны методы сохранения и загрузки дней занятий.

```

458 void MainWindow::onDeleteSet() {
459     if (currentSetName.isEmpty()) {
460         QMessageBox::warning(this, "Ошибка", "Выберите набор для удаления!");
461         return;
462     }
463
464     if (currentSetName == "АНГЛИЙСКИЙ -> РУССКИЙ (30 слов)" || currentSetName == "РУССКИЙ -> АНГЛИЙСКИЙ (30 слов)") {
465         QMessageBox::warning(this, "Ошибка", "Нельзя удалить базовый набор!");
466         return;
467     }
468
469     QMessageBox msgBox;
470     msgBox.setWindowTitle("Удалить набор");
471     msgBox.setText(QString("Вы уверены, что хотите удалить набор '%1'?" ).arg(currentSetName));
472     msgBox.setStandardButtons(QMessageBox::Yes | QMessageBox::No);
473     msgBox.setButtonText(QMessageBox::Yes, "Да");
474     msgBox.setButtonText(QMessageBox::No, "Нет");
475
476     int reply = msgBox.exec();
477
478     if (reply == QMessageBox::Yes) {
479         QFile::remove(currentSetName + ".txt");
480
481         for (int i = 0; i < setList->count(); i++) {
482             QListWidgetItem* item = setList->item(i);
483             if (item->text() == currentSetName) {
484                 delete setList->takeItem(i);
485                 break;
486             }
487         }
488
489         currentCards.clear();
490         activeCards.clear();
491         currentSetName = "";
492         lastCardIndex = -1;
493         questionLabel->setText("Выберите набор для изучения");
494         answerInput->setEnabled(false);
495         checkBtn->setEnabled(false);
496         dontKnowBtn->setEnabled(false);
497         finishBtn->setEnabled(false);
498         deleteBtn->setEnabled(false);
499         updateProgress();
500         QMessageBox::information(this, "Успех", "Набор удален!");
501     }
502 }

```

Рисунок 22- Метод onDeleteSet()- удаление пользовательского набора

Перед удалением метод проверяет, выбран ли набор, и запрещает удаление базовых наборов (английского и русского). Затем выводится диалог подтверждения с кнопками «Да» и «Нет». При подтверждении файл набора удаляется с помощью QFile::remove(), набор удаляется из списка в интерфейсе, а все переменные текущего набора сбрасываются. Базовые наборы защищены от случайного удаления.

На рисунке 23 показаны методы сохранения и загрузки дней занятий.

```

563 void MainWindow::saveDaysData() {
564     QDate today = QDate::currentDate();
565     QString lastDateStr;
566     int currentDays = 0;
567
568     QFile file("days_data.txt");
569     if (file.open(QIODevice::ReadOnly)) {
570         QTextStream in(&file);
571         if (!in.atEnd()) {
572             currentDays = in.readLine().toInt();
573         }
574         if (!in.atEnd()) {
575             lastDateStr = in.readLine();
576         }
577         file.close();
578     }
579
580     QDate lastDate = QDate::fromString(lastDateStr, Qt::ISODate);
581
582     if (lastDate != today) {
583         currentDays++;
584
585         if (file.open(QIODevice::WriteOnly)) {
586             QTextStream out(&file);
587             out << currentDays << "\n";
588             out << today.toString(Qt::ISODate) << "\n";
589             file.close();
590         }
591     }
592 }
593
594 void MainWindow::loadDaysData() {
595     QFile file("days_data.txt");
596     if (!file.exists()) {
597         if (file.open(QIODevice::WriteOnly)) {
598             QTextStream out(&file);
599             out << "0\n";
600             out << QDate::currentDate().toString(Qt::ISODate) << "\n";
601             file.close();
602         }
603     }
604 }

```

Рисунок 23- Методы saveDaysData() и loadDaysData()- сохранение и загрузка дней занятий

Метод saveDaysData() вызывается в конструкторе MainWindow при запуске программы. Он считывает из файла days_data.txt сохранённое

количество дней и дату последнего входа. Если последняя сохранённая дата отличается от сегодняшней, счётчик дней увеличивается на 1, и новые данные записываются в файл. Метод loadDaysData() создаёт файл со стартовыми значениями, если он не существует. Это позволяет отслеживать регулярность занятий: сколько уникальных дней пользователь запускал программу.

Вывод по разделу 2. В практической части разработана программа «Флеш-карточки» для изучения иностранных слов. Построены UML-диаграммы, отражающие структуру приложения. Реализован графический интерфейс на Qt, включающий панель управления, карточку с вопросом, статистику и игровой режим. Разработан алгоритм умного повторения: слова с ошибками появляются чаще, после 7 правильных ответов считаются выученными. Создана игра «Быстрый перевод» с ограничением времени 10 секунд.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта разработано автоматизированное рабочее место специалиста по изучению иностранных слов с использованием метода флеш-карточек.

В теоретической части проведён анализ психологических основ запоминания информации, рассмотрены принципы активного вспоминания и интервального повторения. Выполнен обзор и сравнительный анализ существующих программ (Anki, Quizlet, Memrise, Lingualeo), выявлены их недостатки: сложность интерфейса, платная подписка, зависимость от интернета. Обоснован выбор языка C++ и фреймворка Qt как оптимальных технологий для разработки.

В практической части реализованы все компоненты автоматизированного рабочего места. Графический интерфейс содержит список наборов, карточку с вопросом, поле для ввода ответа, кнопки управления, блок статистики с прогресс-баром и счётчиком дней занятий, а также игровой режим «Быстрый перевод». Разработан алгоритм умного повторения, основанный на взвешенном случайном выборе: слова с большим количеством ошибок появляются чаще, а после 7 правильных ответов считаются выученными. Реализована игра с ограничением времени 10 секунд на каждое слово. Организовано сохранение пользовательских наборов в текстовые файлы и учёт дней занятий для отслеживания регулярности обучения. Построены UML-диаграммы классов и последовательности, отражающие архитектуру приложения.

Все поставленные задачи выполнены. Разработанное автоматизированное рабочее место позволяет пользователю эффективно запоминать иностранные слова, отслеживать свой прогресс и закреплять материал в игровой форме. Программа полностью бесплатна, работает без интернета и имеет простой интуитивно понятный интерфейс.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Эббингауз Г. Память и её законы. Исследования по психологии запоминания. URL: <https://studfile.net/preview/9418823/page:33/> (дата обращения: 20.05.2026)
2. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления.- URL: https://www.hse.ru/data/2020/10/06/1370744192/ГОСТ_7_32_2017_Отчёт_по_НИР_с_выделением.pdf (дата обращения: 20.05.2026)
3. Официальная документация Qt 5.12.- URL: <https://doc.qt.io/qt-5.12/> (дата обращения: 20.05.2026)
4. Официальная документация Anki.- URL: <https://docs.ankiweb.net/> (дата обращения: 20.05.2026)
5. Официальная документация Quizlet.- URL: <https://quizlet.com/help> (дата обращения: 20.05.2026)
6. Флеш-карточки: техника обучения, которая поможет улучшить карьеру и жизнь.- URL:<https://brainapps.ru/blog/2023/10/flesh-kartochkia-tehnika-obucheniya-kotoraya/> (дата обращения: 20.05.2026)
7. Официальная документация OBS Studio.- URL: <https://obsproject.com/wiki/> (дата обращения: 20.05.2026)